

Calculated columns with DAX

An introduction to DAX

DAX: The language of data in Power BI

Data Analysis Expressions (DAX) is a **programming language** that is used in **Power BI** to **create** calculated **columns**, **measures**, and custom **tables**.

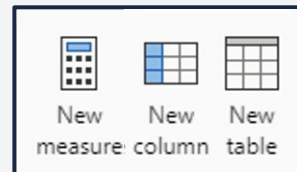
It is a **collection** of **functions**, **operators**, and **constants** that can be used in formulas to perform advanced calculations and data analysis.

It allows us to write custom calculations using the **formula bar** and **return one or more values**.



How to input DAX in Power BI

We **input** DAX code by selecting the **Modeling** tab in **Report view** or the **Table tools** tab in **Table view** and choosing the appropriate option:



A **formula bar** pops up where we can type our DAX code:





Composing formulas in DAX

Every **DAX formula** is a combination of **functions**, **operators**, and **constants**. Understanding how these **work together** is key to mastering DAX.

Functions

Predefined formulas that **perform specific calculations** using the data in our tables, such as adding up numbers or finding an average.

Operators

Symbols or **keywords** that specify the **type of calculation** to perform between elements in a formula, like adding with "+" or multiplying with "*".

Constants

Fixed values that we directly enter into our formulas, such as a number, a text string, or a date.

Using DAX with data

We will use a subset of the **Gender_parity_2022** dataset to illustrate how we use DAX.

Country	Percent_literacy_women	Percent_literacy_men	GDP_per_capita_USD	Population_size
Egypt	67.38999939	78.77999878	4,295.40	110,990,103
Ethiopia	44.42338181	59.24145889	1,027.60	123,379,924
Ghana	76.19000244	84.48999786	2,175.90	33,475,870
Kenya	79.84999847	85.48000336	2,099.30	54,027,487
Morocco	67.37999725	84.80999756	3,527.90	37,457,971
Rwanda	73.30000305	78.69999695	966.3	13,776,698
Nigeria	52.65647888	71.25570679	2,184.40	218,541,212
South Africa	94.53317261	95.54537201	6,776.50	59,893,885

Calculated columns

A calculated column is a **new column** that we add to a table that is **computed from other columns**.

Unlike a regular column, a calculated column **uses a DAX formula** that **defines its values**. These are **stored** in the model and can be used in reports just like any other column.

*This isn't just about displaying new information, it's about **creating** it from what we **already have**.*



To understand the balance between female and male literacy rates, we could create a **calculated column** that **stores** the **difference** between these values.



New
column

DAX: Literacy_difference = 'Gender_parity_2022'[Percent_literacy_women]-
'Gender_parity_2022'[Percent_literacy_men]

Literacy_difference

-11.38999939

-14.81807708

-8.29999542

-5.63000489

-17.43000031

-5.3999939

-18.59922791

-1.0121994

How are values in the calculated column determined?

Row context tells DAX which row to use when **determining the values** for the calculated column.

We can think of the row context as the **current row** under calculation in a table. It's like having a magnifying glass that looks at each row one by one during the calculation.



For example, DAX used row context to subtract the value of Percent_literacy_men from Percent_literacy_women, **row by row**, and then stored the result in the **same row** in the Literacy_difference column. The key to understanding the row context is in that "**row by row**".

E.g., Use the values in this row



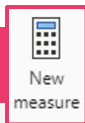
Percent_literacy_women	Percent_literacy_men	Literacy_difference
67.38999939	78.77999878	-11.38999939

To determine the value in the same row in the calculated column

Other places we can use DAX

In addition to calculated columns, DAX also empowers us to create **measures** and **tables**, to further analyse and summarise our data.

Measure



- Measures are **calculations** created using DAX that are computed **dynamically** based on the **current report context**.
- They are often used to perform **summarisation operations** like sum, average, or count, that work across the **entire table** or within the filters set by users, not just row by row.
- Unlike calculated columns, measures **do not store their results** in the database – they're re-calculated whenever needed.

Table



- Custom tables can be created using DAX to **organise data** in **new ways** or **summarise** it for **specific views**.
- They allow us to **combine data** from **multiple sources** and **store** it within the Power BI model.
- These tables can include **calculated columns** for specific analysis needs, and **measures** can be used **alongside** these tables to provide dynamic summaries.

Choosing between a calculated column or a measure

	Calculated column	Measure
Purpose	Used to add new data to an existing table, expanding its detail .	Used to aggregate data , providing high-level summaries and insights.
Evaluation	Created using row context and is computed during data refresh, becoming a permanent part of the table.	Evaluated when a query is made based on the current filters we have set, allowing dynamic calculation based on the current view .
Storage	Becomes part of the table's storage (in import mode) and can influence report size and performance .	Doesn't store data – it calculates on demand and is typically more efficient for summarisation .
Visual use	Can be used like any other column to sort, filter, and group data in reports.	Typically used in visuals to present aggregated data , such as totals or averages.

Functions in Power BI

DAX functions

Functions are **prewritten formulas** that we can use to **perform operations on data**. They can be as simple as adding two numbers or as complex as filtering a table based on multiple conditions.

In DAX, these functions are enhanced by the ability to **reference various model objects** like tables, columns, and measures, enabling refined and powerful data calculations within our Power BI reports.

For example, to analyse literacy rate trends by gender, we could use the AVERAGE function on Percent_literacy_women and Percent_literacy_men.



```
Average_women_literacy_rate = AVERAGE('Gender_parity_2022'[Percent_literacy_women])
```

```
Average_men_literacy_rate = AVERAGE('Gender_parity_2022'[Percent_literacy_men])
```

Referencing model objects

In DAX, we can **reference entire tables**, specific **columns** within tables, or **measures** that we've already created. This allows for a **modular approach** where measures can build on each other and on calculated columns.

Table references

Refers to the **entire table** within the data model. Used in functions that need to consider **the entire dataset**.

Syntax:

`'Table_name'`

Column references

Refers to **specific columns** within a table. Used when we need to perform operations on **particular fields**, such as creating calculated columns or defining criteria in filters.

Syntax:

`'Table_name' [Column_name]`

Measure references

Refers to **measures** that have been defined in the model. Allows the **reuse** of measures within other DAX formulas, maintaining **consistency** and **reducing redundancy** in calculations.

Syntax:

`[Measure_name]`

Bringing it all together

Let's create a measure that calculates the average GDP for countries with a female literacy rate above 70%.

1. First, we define a measure called `Average_GDP_per_capita` to calculate the average GDP.

```
Average_GDP_per_capita = AVERAGE ( 'Genderparity_2022' [GDP_per_capita_USD] )
```

Table reference

Column reference

Function

2. Next, we create a measure that calculates the average GDP for countries with a female literacy rate above 70%.

```
Avg_GDP_for_high_female_literacy =
```

```
    CALCULATE (
        [Average_GDP_per_capita],
        'Genderparity_2022' [Percent_literacy_women] > 70
    )
```

Function

Measure reference

Filter

Column reference